Software Requirements & Design Constraints

CMP9134: Software Engineering

Dr Francesco Del Duchetto
Lecturer in Robotics and Autonomous Systems

University of Lincoln

20 February 2026

Today's Agenda

1 What is Software Requirements Engineering?

2 LEPSI as Design Constraints

3 Stages of the Requirements Process

4 Requirements in Agile Scrum

5 Next Steps

Today's Learning Objectives

By the end of this lecture, you should be able to:

1. **Define** the purpose of Software Requirements Engineering and distinguish between Functional and Non-Functional Requirements (NFRs).
2. **Recognise** Legal, Ethical, Professional, and Social Issues (LEPSI) not as abstract concepts, but as strict design constraints and NFRs.
3. **Identify** how regulatory compliance (e.g., GDPR), privacy by design, and accessibility (e.g., WCAG) dictate software architecture choices.
4. **Apply** Agile methodologies to capture LEPSI constraints using User Stories and a robust "Definition of Done" (DoD).

Agenda

1 What is Software Requirements Engineering?

2 LEPSI as Design Constraints

3 Stages of the Requirements Process

4 Requirements in Agile Scrum

5 Next Steps

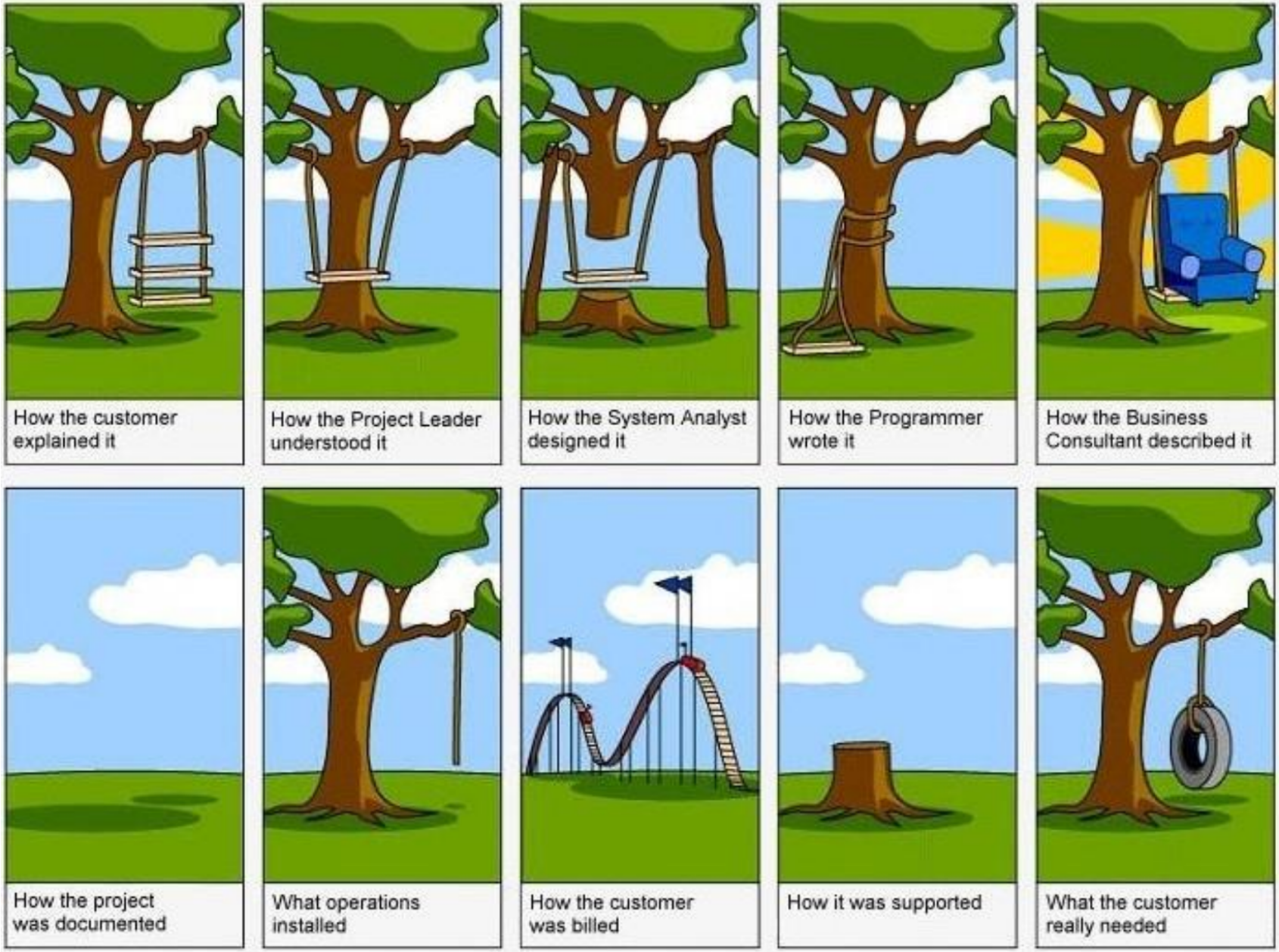
What are Requirements?

Definition

Software Requirements Engineering is the process of identifying, documenting, and maintaining software requirements.

Requirements describe **how a software system should behave** or the **attributes it must have**.

- They ensure developers and stakeholders share a common understanding of what the software should do.
- *A project without clear requirements is guaranteed to fail.*


How the customer explained it
How the Project Leader understood it
How the System Analyst designed it
How the Programmer wrote it
How the Business Consultant described it
How the project was documented
What operations installed
How the customer was billed
How it was supported
What the customer really needed

Functional vs. Non-Functional

Functional Requirements

What the system should DO.

- Specific functions, commands, data processing.
- E.g., ”The system shall allow the user to log in.”
- E.g., ”The robot must accept target coordinates.”

Non-Functional Requirements

HOW the system should do it.

- Quality attributes, performance, security, constraints.
- E.g., ”The system must process the login in under 2 seconds.”
- E.g., ”The robot API connection must handle dropouts.”

Non-Functional Requirements (NFRs)

NFRs are often overlooked but are critical for the success of a software project. Non-Functional Requirements can be divided into three main types:

- **Product Requirements:** Performance, Reliability, Usability.
- **Organizational Requirements:** Development standards, Delivery processes.
- **External Requirements:** Ethical, Legislative, Accounting, Interoperability.

Metrics for Non-Functional Requirements

Property	Measure
Speed	Processed transactions/second, Response time
Size	Mbytes, Storage footprint
Ease of use	Training time, Number of help frames
Reliability	Mean time to failure, Probability of unavailability
Robustness	Time to restart after failure, Data corruption probability
Portability	Target dependent statements, Number of target systems

Agenda

1 What is Software Requirements Engineering?

2 LEPSI as Design Constraints

3 Stages of the Requirements Process

4 Requirements in Agile Scrum

5 Next Steps

LEPSI in Design

The Developer's Trap

"Requirements are just the features the user asked for."

Before you write a single line of code, you must realize that **Legal, Ethical, Professional, and Social Issues (LEPSI)** are not abstract concepts—they are **Non-Functional Requirements (NFRs)**.

- **Legal** $\rightarrow$ Regulatory Compliance (e.g., GDPR, CCPA).
- **Social** $\rightarrow$ Accessibility (e.g., WCAG).
- **Professional** $\rightarrow$ Ethical Data Collection & Privacy by Design.

Regulatory Compliance as a Requirement

Your system architecture is dictated by law.

- **GDPR (EU,UK) / CCPA (California):** Governs the protection of personal data.
- **Design Constraints imposed:**
  - You must obtain consent before collecting data.
  - You must implement the ”Right to be Forgotten” (deletion workflows).
  - *Architecture impact:* User data cannot be permanently baked into immutable logs.
- **HIPAA (US Health) / PCI DSS (Payments):** Requires strict encryption, access controls (RBAC), and vulnerability scanning.

Privacy & Cybersecurity by Design

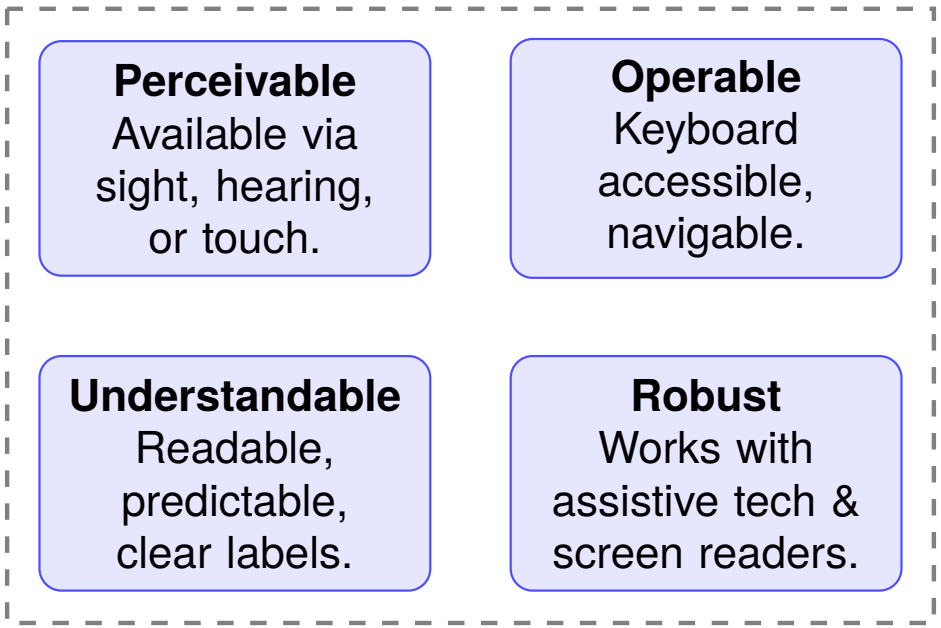
Privacy is protecting personal information. **Security** is protecting the systems from unauthorized access.

How to map these to Requirements:

- *Req 1:* ”The system must encrypt all user passwords using Bcrypt.” (Security)
- *Req 2:* ”The system shall only collect telemetry data strictly necessary for the mission.” (Privacy/Data Minimisation)
- *Req 3:* ”The system must feature Role-Based Access Control (RBAC) to separate ’Viewers’ from ’Commanders’.” (Security Constraint).

Accessibility as a Requirement (WCAG)

Accessibility ensures software is usable by all people, including those with disabilities. The W3C defines **POUR** principles:



AI Ethics & Dual Use

If you integrate AI or build autonomous systems (like the Robot Management System in Assessment 1), new ethical constraints emerge.

- **Explainability (XAI):** Can you explain why the robot made a specific decision? (Rule-based vs Black-box models).
- **Safety:** What happens when network latency causes the robot to miss a ”Stop” command? (Failsafes are NFRs).
- **Dual Use:** Could the software you are building be used for malicious purposes if hijacked?

Agenda

1 What is Software Requirements Engineering?

2 LEPSI as Design Constraints

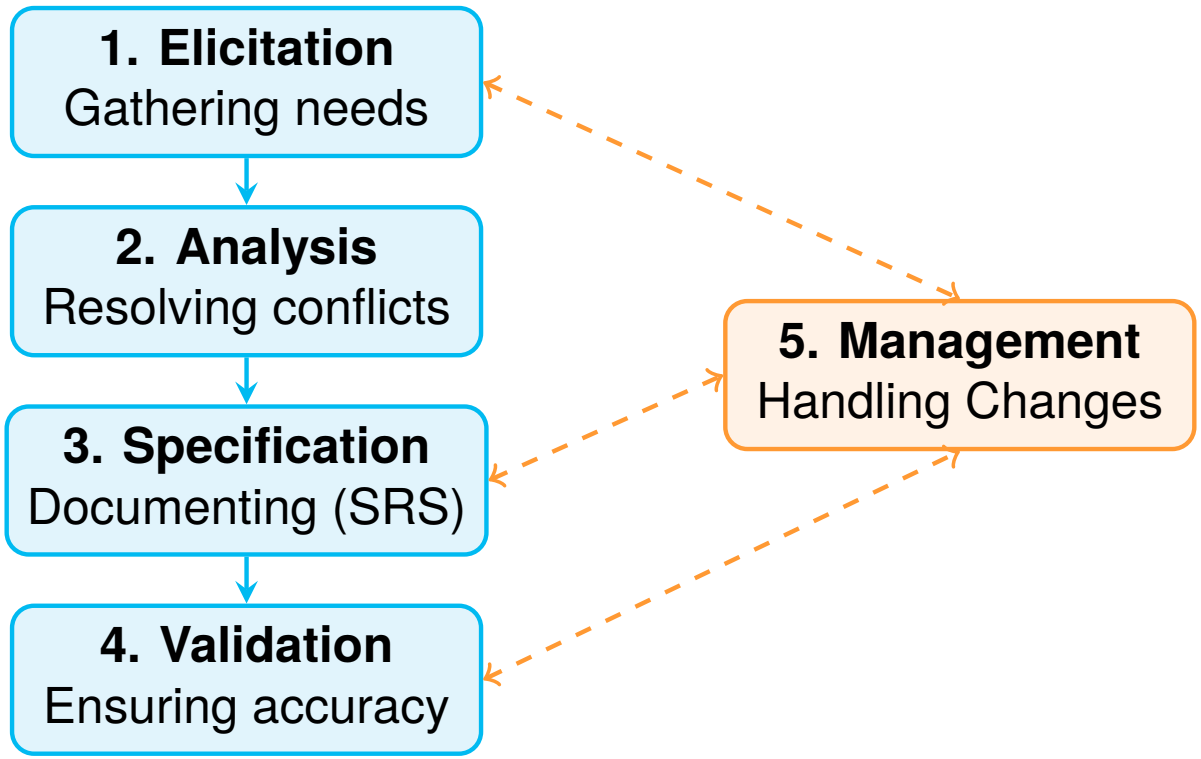
3 Stages of the Requirements Process

4 Requirements in Agile Scrum

5 Next Steps

The Requirements Engineering Process

The process involves several steps, including requirements elicitation, analysis, specification, and validation, to ensure that the software will perform as intended.



1. Elicitation - Tools

This is the process of gathering requirements from stakeholders, users, and other sources. It can involve:

- **Interviews and questionnaires**.
- **Workshops and brainstorming** sessions.
- **Observation** of current systems and user behavior.
- Analysis of **existing documentation** and **market research**.

2. Analysis - Tools

After gathering requirements, you need to analyze them to resolve conflicts, prioritize features, and ensure they are feasible.

- **Use Case Diagrams** to visualize interactions.
- **Data Flow Diagrams** to understand data movement.
- **Requirement Prioritization** techniques (MoSCoW, Kano).
- **Feasibility Analysis** to assess technical and economic viability.

3. Specification - Tools

The Software Requirements Specification (SRS) document serves as a blueprint for development.

Key Elements of an SRS

1. **Purpose** of the software.
2. **Overall description** of the system.
3. System **functionality** (Functional Req).
4. **External Interfaces** (API connections).
5. **Non-functional requirements** (Performance, Security).
6. **Design Constraints** (LEPSI, Legal limits, Operating Environment).

4. Validation - Tools

Validation ensures that the specified requirements are correct, complete, and meet stakeholder needs.

- **Reviews and Inspections** of the SRS document.
- **Prototyping** to gather early feedback.
- **Test Case Generation** to verify requirements can be tested.
- **Traceability Matrices** to ensure all requirements are covered by tests.

5. Management - Tools

Managing requirements involves tracking changes, ensuring consistency, and maintaining control over the evolving scope of a project.

- **Change Request Management** to track and approve changes.
- **Version Control** systems to manage document versions.
- **Requirements Traceability** to link requirements to design and testing artifacts.
- **Configuration Management** to maintain system integrity across versions.

Agenda

1 What is Software Requirements Engineering?

2 LEPSI as Design Constraints

3 Stages of the Requirements Process

4 Requirements in Agile Scrum

5 Next Steps

Agile Requirements: User Stories

In Agile, requirements are captured as **User Stories** stored in the **Product Backlog**.

Format: ”As a **[User]**, I want **[Functionality]**, so that **[Benefit]**.”

Example (Applying LEPSI): *”As a system auditor, I want all navigation commands to be logged persistently, so that we comply with safety accountability regulations.”*

Agile Requirements: Acceptance Criteria

Each User Story must be accompanied by **Acceptance Criteria** (AC) that define the conditions under which the story is considered complete.

Purpose of AC:

- Define clear expectations for the feature.
- Ensure developers and stakeholders are aligned.
- Provide testable conditions for validation.

Definition of Done (DoD)

How do we know a requirement is actually met? We use a Definition of Done.

To consider a User Story ”Done”, a team might agree it must pass the following checklist:

- ☐ **Code compiles and runs.**
- ☐ **Unit tests pass.**
- ☐ **Security scan passes** (No hardcoded passwords).
- ☐ **UI meets WCAG accessibility guidelines.**
- ☐ ...

Practical Agile: Writing Good User Stories

How do you write a User Story for your Robot Management System? Avoid vague feature requests.

Poor User Story

”Make the robot move.” *(Too vague, lacks context, no clear user or benefit).*

Strong User Story

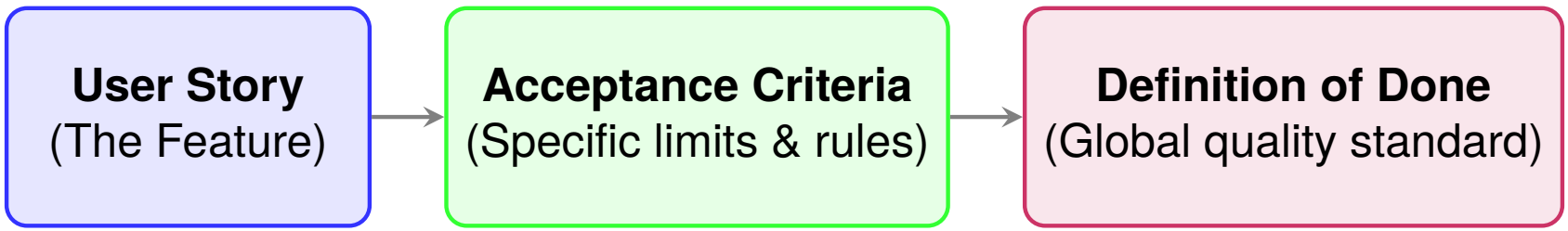
”As a **Commander**, I want to **input X and Y coordinates into the dashboard**, so that **the robot navigates to the target location**.”

Why is this better?

- It defines **who** needs it (Commander, not a Viewer).
- It defines **what** the interface needs (X and Y inputs).
- It defines the **business value** (navigating to a target).

Acceptance Criteria vs. Definition of Done

While the **Definition of Done (DoD)** applies globally to *every* task, **Acceptance Criteria (AC)** are specific to a *single* user story.



- **Example AC for Robot Navigation:**
  - The UI must have two numeric input fields (X and Y).
  - The system must reject negative coordinates (Error Handling).
  - A POST request must be sent to the API's `/move` endpoint.
  - **LEPSI Constraint:** The "Move" button must be accessible via Keyboard (WCAG).

Agenda

1 What is Software Requirements Engineering?

2 LEPSI as Design Constraints

3 Stages of the Requirements Process

4 Requirements in Agile Scrum

5 Next Steps

Any Questions?

fdelduchetto@lincoln.ac.uk